GLITCH

Guía breve para entender y explicar el funcionamiento del código

Esta guía resume cómo funciona el proyecto sin explicar cada línea. El objetivo es poder estudiar la lógica general y contarla con claridad en una exposición.

1. ¿Cómo funciona el juego?

GLITCH es un juego de supervivencia y persecución. El jugador debe mantenerse con vida mientras una anomalía lo persigue dentro de arenas que cambian durante la partida.

El jugador puede moverse, hacer Ghost Dash, rechazar ataques con Firewall Parry, liberar un Firewall Burst y recoger powerups. Mientras sobrevive suma puntos, completa objetivos y atraviesa Breaches para pasar a sectores nuevos. La partida termina cuando recibe un impacto que no logra evitar.

La idea principal del código es separar responsabilidades: un script coordina la partida, otros controlan al jugador y al enemigo, y otros generan el mapa, los eventos y la interfaz.

2. Organización general

Los sistemas más importantes son los siguientes:

Script o sistema	Responsabilidad
`GameManager`	Coordina el inicio, el desarrollo y el final de la run.
`PlayerController`	Lee los controles y ejecuta las habilidades del jugador.
`EnemyController`	Persigue al jugador y administra los estados de la anomalía.
`ProceduralArenaGenerator`	Construye la arena y cambia el tipo de sector.
`ArenaChaosDirector`	Administra pickups, objetivos y eventos generales.
Controladores de cada mapa	Ejecutan eventos propios de Lab, Core, Rupture, Storage y Archive.
`GameMenuController`	Dibuja el HUD, la pausa y la pantalla de derrota.
`MainMenuController`	Maneja el menú principal, las opciones y la selección de clase.
Sistemas `Storage`	Guardan progresión, preferencias y estadísticas locales.

La relación puede resumirse así:

```
Menú principal
      ↓
Selección de modo y clase
      ↓
Generación del mapa
      ↓
GameManager inicia la partida
      ├── PlayerController controla al jugador
      ├── EnemyController controla a la anomalía
      └── Directores de eventos modifican la arena
      ↓
Breach para continuar o derrota para finalizar
```

3. Inicio de una partida

El jugador comienza en el menú principal. Allí puede elegir el modo de juego, una clase y las opciones de audio o interfaz. Si todavía no registró un nombre válido, el menú muestra una ventana obligatoria para usarlo en el ranking global.

Cuando se carga gameplay, `ProceduralArenaGenerator` crea los límites, los obstáculos y la estética del sector. También agrega los controladores necesarios para los eventos de ese mapa.

Después, `GameManager` prepara la run: reinicia puntaje y tiempo, aplica la clase elegida, activa la telemetría y decide qué mecánicas están disponibles. Finalmente muestra una cuenta regresiva breve y habilita el movimiento.

Los tutoriales no explican todos los contratos por separado. Presentan los controles y la interfaz, y luego muestran una pausa contextual solamente la primera vez que aparece una mecánica o un evento nuevo.

4. Jugador y anomalía

Jugador

`PlayerController` consulta el teclado y el mouse en cada cuadro. A partir de esa entrada mueve al personaje o intenta ejecutar una habilidad.

- El movimiento usa WASD o las flechas.
- Ghost Dash produce un desplazamiento rápido y luego entra en enfriamiento.
- Firewall Parry rechaza una amenaza si se usa en el momento correcto.
- Un Parry exitoso carga el Firewall.
- Con la carga completa, Firewall Burst libera una defensa alrededor del jugador.
- Los powerups agregan efectos temporales, como escudo, velocidad, tamaño reducido o Phase Dash.

El controlador también informa su estado al HUD. Por eso la interfaz puede mostrar enfriamientos, carga de Firewall, powerups activos y señales visuales sobre el propio personaje.

Anomalía

`EnemyController` busca una ruta hacia el jugador y ajusta su movimiento para rodear obstáculos. Su comportamiento base es perseguir, pero puede activar estados especiales que cambian las reglas durante algunos segundos.

Entre esos estados puede acelerar, dividirse, disparar, atravesar o destruir obstáculos, teletransportarse o comportarse de manera errática. El juego desbloquea estos estados progresivamente para que la dificultad crezca durante la run.

El enemigo también se relaciona con la defensa del jugador. Un proyectil puede ser reflejado con Parry y ciertos estados pueden ser capturados mediante State Hijack. De esta manera, defenderse también crea oportunidades de contraataque.

5. Mapas, eventos y progresión de la run

La arena no es un fondo fijo. Cada sector tiene una distribución procedural y un grupo de eventos propios.

- Lab utiliza terminales y sistemas de seguridad.
- Core puede activar peligros ambientales, como objetos con espinas.
- Rupture modifica y mueve los objetos de la arena.
- Storage y Archive poseen sus propias reglas visuales y jugables.
- Los eventos globales pueden aparecer en más de un mapa.

`ArenaChaosDirector` organiza el contenido general y evita que todas las amenazas importantes se activen sin control. Los controladores particulares de cada mapa se ocupan de su comportamiento específico.

Durante la partida también aparecen contratos, objetivos, pickups y mejoras. Completar estas acciones entrega puntos o recursos. La clase elegida modifica el estilo de juego: puede favorecer ataque, defensa, obtención de recompensas o una experiencia mucho más caótica.

Cuando aparece una Breach comienza una secuencia de escape. El jugador debe alcanzar el portal antes de que el barrido de la arena lo alcance. Si lo consigue, se genera un sector nuevo y la run continúa con mayor dificultad.

6. Final, guardado y ranking

Cuando el jugador pierde, `GameManager` detiene la acción y `GameMenuController` muestra el resultado. El juego calcula una clasificación final —S, A, B, C o D— según el puntaje obtenido.

En ese momento se registran datos útiles de la run, como duración, nivel alcanzado, puntaje, eventos y causa de derrota. La telemetría se guarda localmente para poder analizar dificultad, errores y nivel de participación sin realizar escrituras constantes durante la partida.

La progresión local usa sistemas de almacenamiento propios y `PlayerPrefs`. Allí se conservan opciones, tutoriales vistos, desbloqueos y nombre del jugador. Desde Opciones existe un botón para borrar esos datos y comenzar nuevamente desde cero.

El ranking global usa Firebase Realtime Database. Cada instalación se autentica y posee un identificador propio. El juego consulta una lista limitada de mejores resultados y solo escribe cuando el nuevo puntaje supera el récord anterior del mismo jugador. Esto reduce lecturas y escrituras innecesarias.

7. Idea técnica importante

Unity ejecuta automáticamente ciertos métodos de los componentes:

- `Awake` prepara referencias y valores internos.
- `Start` inicia el sistema cuando la escena ya está cargada.
- `Update` actualiza controles, relojes y estados en cada cuadro.
- Las coroutines permiten realizar secuencias con esperas, como avisar un evento, activarlo y terminarlo.
- `Rigidbody2D` y `Collider2D` resuelven movimiento, física y contactos.

No existe un único script que haga todo. Los controladores se consultan entre sí o reciben avisos cuando ocurre algo. Por ejemplo, un Parry comienza en `PlayerController`, afecta a la amenaza, aumenta la carga de Firewall, reproduce sonido y actualiza el HUD.

Esta separación permite modificar una parte del juego sin reescribir todas las demás. También facilita detectar errores: si falla la persecución se revisa el enemigo; si falla un evento de Lab se revisa su controlador; y si el problema aparece al iniciar o terminar una run se revisa `GameManager`.

8. Resumen para explicar al profesor

Una explicación breve puede ser:

> **GUION SUGERIDO**
>
> "Organicé el proyecto por responsabilidades. `GameManager` dirige la partida; `PlayerController` y `EnemyController` manejan a los dos personajes principales; el generador construye los sectores; y distintos directores agregan eventos, objetivos y recompensas. Durante la run estos sistemas comparten información para aumentar la dificultad y actualizar la interfaz. Al perder, el juego guarda progresión y telemetría, calcula una clasificación y actualiza Firebase solamente si el jugador superó su mejor puntaje."

Las cuatro ideas que conviene recordar son:

1. Cada sistema tiene una responsabilidad concreta.
2. `GameManager` coordina el flujo general, pero no ejecuta todas las mecánicas.
3. El jugador, la anomalía, el mapa y los eventos interactúan durante toda la run.
4. El guardado local conserva progresión; Firebase conserva únicamente el mejor puntaje global.